

COMP 576 Assignment 1

Lingyi Xu (lx28@rice.edu)

Oct 6th, 2024

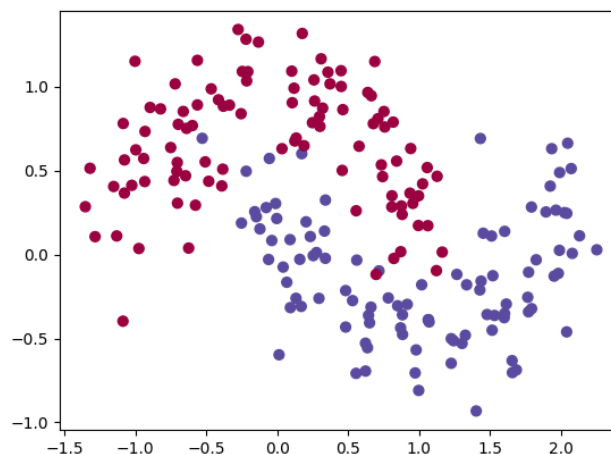
Table of Contents

1. [Backpropagation in a Simple Neural Network](#)
 - [1. a\) Dataset](#)
 - [1. b\) Activation Function](#)
 - [1. c\) Build the Neural Network](#)
 - [1. d\) Gradients](#)
 - [1. e\) Time to Have Fun - Training!](#)
 - [1. f\) Even More Fun - Training a Deeper Network!](#)
2. [Training a Simple Deep Convolutional Network on MNIST](#)
 - [2. a\) Build and Train a 4-layer DCN](#)
 - [2. b\) Visualizing Training](#)
 - [2. c\) Leaky ReLU + Xavier Initialization + SGD](#)

1. Backpropagation in a Simple Neural Network

1. a) Dataset

Uncomment the "generate visualize Make-Moons dataset", we get:



1 b) Activation Function

1.b.1 Implement function `actFun(self, z, type)` in `three_layer_nerual_network.py` :

```
def actFun(self, z, type):  
    """  
    actFun computes the activation functions  
    :param z: net input  
    :param type: Tanh, Sigmoid, or ReLU  
    :return: activations  
    """  
    # YOU IMPLMENT YOUR actFun HERE  
    if type.lower() == 'tanh':  
        return np.tanh(z)  
    if type.lower() == 'sigmoid':  
        return 1 / (1 + np.exp(-z))  
    elif type.lower() == 'relu':  
        return np.maximum(0, z)  
    else:  
        raise ValueError('Activation function type must be tanh, Sigmoid, or ReLU.')
```

1.b.2 Derive the derivatives of Tanh, Sigmoid and ReLU

- The derivate of $\tanh(z)$ is $1 - \tanh(z)^2$
- The derivate of $\text{sigmoid}(z)$ is $\sigma(z)(1 - \sigma(z))$
- The derivate of $\text{ReLU}(z)$ is 1 for $z > 0$, and 0 otherwise.

1.b.3 Implement function `diff_actFun(self, z, type)`

```
# YOU IMPLEMENT YOUR diff_actFun HERE  
if type.lower() == 'tanh':  
    return 1 - np.tanh(z) ** 2  
elif type.lower() == 'sigmoid':  
    sig = 1 / (1 + np.exp(-z))  
    return sig * (1 - sig)  
elif type.lower() == 'relu':  
    return np.where(z > 0, 1, 0)  
else:  
    raise ValueError("Unsupported activation function type.")
```

1 c) Build the Neural Network

1.c.1 implement the function `feedforward(self, X, actFun)`

```
def feedforward(self, X, actFun):  
    ...  
    feedforward builds a 3-layer neural network and computes the probabilities  
    :param X: input data  
    :param actFun: activation function  
    :return: None  
    ...  
  
    self.z1 = X.dot(self.W1) + self.b1  
    self.a1 = actFun(self.z1)  
    self.z2 = self.a1.dot(self.W2) + self.b2  
    exp_scores = np.exp(self.z2)  
    self.probs = exp_scores / np.sum(exp_scores, axis=1, keepdims=True)
```

1.c.2 Fill in the function `calculate_loss(self, X, y)`

Used the cross-entropy loss function:

$$L = -\frac{1}{N} \sum_{n=1}^N \sum_{i=1}^C y_{n,i} \log \hat{y}_{n,i}$$

```
# Calculating the loss  
correct_logprobs = -np.log(self.probs[range(num_examples), y])  
data_loss = np.sum(correct_logprobs)
```

1 d)

1.d.1 Derive the following gradients mathematically

$$\delta_3 = probs \delta_3[range(N), y] - 1$$

Gradients for W_2 and b_2 :

$$\frac{\partial L}{\partial W_2} = a_1^T \delta_3$$

$$\frac{\partial L}{\partial b_2} = \sum_{n=1}^N \delta_3$$

Backpropagating to the hidden layer:

$$\delta_2 = \delta_3 W_2^T \times \text{diff_actFun}(z_1)$$

Gradients for W_1 and b_1 :

$$\frac{\partial L}{\partial W_1} = X^T \delta_2$$

$$\frac{\partial L}{\partial b_1} = \sum_{n=1}^N \delta_2$$

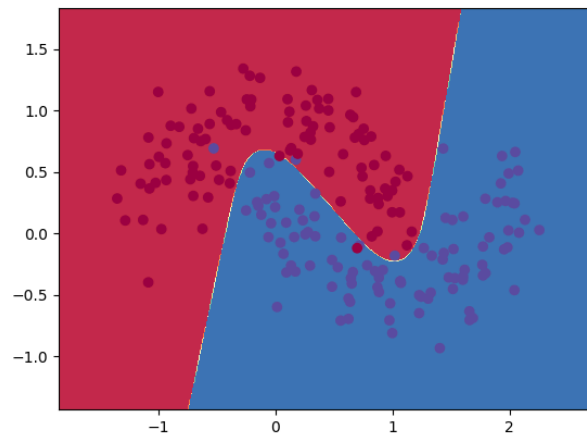
1.d.2 Implement the function `backprop(self, X, y)`

```
dW2 = (self.a1).T.dot(delta3)
db2 = np.sum(delta3, axis=0, keepdims=True)
delta2 = delta3.dot(self.W2.T) * self.diff_actFun(self.z1, self.actFun_type)
dW1 = X.T.dot(delta2)
db1 = np.sum(delta2, axis=0, keepdims=True)
```

1 e) Time to Have Fun - Training!

1.e.1 Train the network using different activation functions (Tanh, Sigmoid and ReLU)

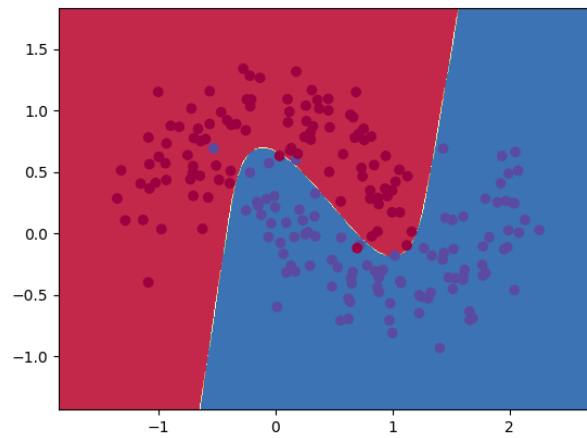
- Tanh:



Output:

```
Loss after iteration 0: 0.432387
Loss after iteration 1000: 0.068947
Loss after iteration 2000: 0.068916
Loss after iteration 3000: 0.070752
Loss after iteration 4000: 0.070748
Loss after iteration 5000: 0.070751
Loss after iteration 6000: 0.070754
Loss after iteration 7000: 0.070756
Loss after iteration 8000: 0.070757
Loss after iteration 9000: 0.070758
Loss after iteration 10000: 0.070758
Loss after iteration 11000: 0.070758
Loss after iteration 12000: 0.070758
Loss after iteration 13000: 0.070758
Loss after iteration 14000: 0.070758
Loss after iteration 15000: 0.070758
Loss after iteration 16000: 0.070758
Loss after iteration 17000: 0.070758
Loss after iteration 18000: 0.070758
Loss after iteration 19000: 0.070758
```

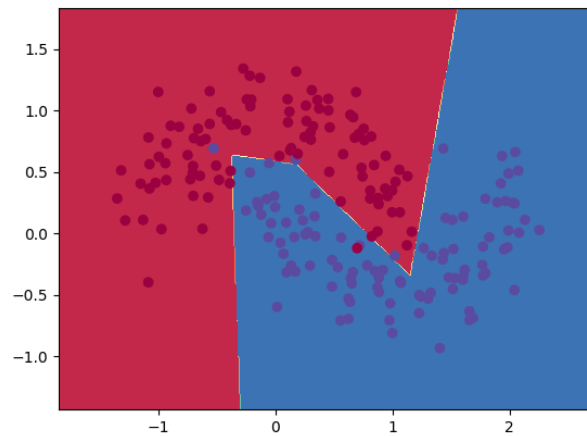
- **Sigmoid:**



Output:

```
Loss after iteration 0: 0.628571
Loss after iteration 1000: 0.088431
Loss after iteration 2000: 0.079598
Loss after iteration 3000: 0.078604
Loss after iteration 4000: 0.078330
Loss after iteration 5000: 0.078233
Loss after iteration 6000: 0.078192
Loss after iteration 7000: 0.078174
Loss after iteration 8000: 0.078166
Loss after iteration 9000: 0.078161
Loss after iteration 10000: 0.078159
Loss after iteration 11000: 0.078158
Loss after iteration 12000: 0.078157
Loss after iteration 13000: 0.078156
Loss after iteration 14000: 0.078156
Loss after iteration 15000: 0.078156
Loss after iteration 16000: 0.078156
Loss after iteration 17000: 0.078156
Loss after iteration 18000: 0.078156
Loss after iteration 19000: 0.078155
```

- **ReLU:**



Output:

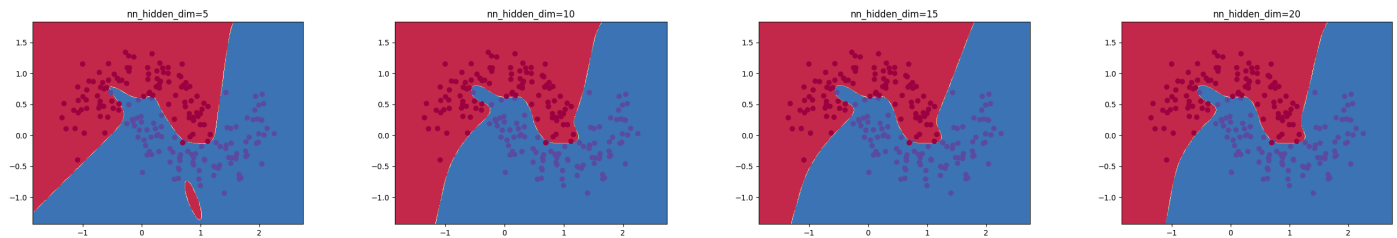
```
Loss after iteration 0: 0.560274
Loss after iteration 1000: 0.072179
Loss after iteration 2000: 0.071301
Loss after iteration 3000: 0.071159
Loss after iteration 4000: 0.071190
Loss after iteration 5000: 0.071136
Loss after iteration 6000: 0.071276
Loss after iteration 7000: 0.071090
Loss after iteration 8000: 0.071265
Loss after iteration 9000: 0.071084
Loss after iteration 10000: 0.071090
Loss after iteration 11000: 0.071087
Loss after iteration 12000: 0.071086
Loss after iteration 13000: 0.071069
Loss after iteration 14000: 0.071114
Loss after iteration 15000: 0.071074
Loss after iteration 16000: 0.071113
Loss after iteration 17000: 0.071071
Loss after iteration 18000: 0.071090
Loss after iteration 19000: 0.071219
```

Difference:

- **Tanh vs. Sigmoid:** Both Tanh and Sigmoid activations provide smooth decision boundaries, reflecting their continuous and differentiable nature. However, Tanh converged slightly faster and to a lower loss value compared to Sigmoid. This is expected, as Tanh has steeper gradients and is centered around zero, which reduces the vanishing gradient problem and speeds up training. Sigmoid, on the other hand, suffers more from vanishing gradients, causing slower convergence and a higher final loss.

- **ReLU:** ReLU produces a more jagged and piecewise-linear decision boundary compared to Tanh and Sigmoid. This is due to ReLU's non-linearity, which only activates for positive inputs, leading to sharper and more angular boundaries. While ReLU often speeds up training due to avoiding the vanishing gradient problem, its decision boundary here does not appear as smooth or well-fitted to the data as those generated by Tanh or Sigmoid.

1.e.2 Increase the number of hidden units (`nn_hidden_dim`)



Differences:

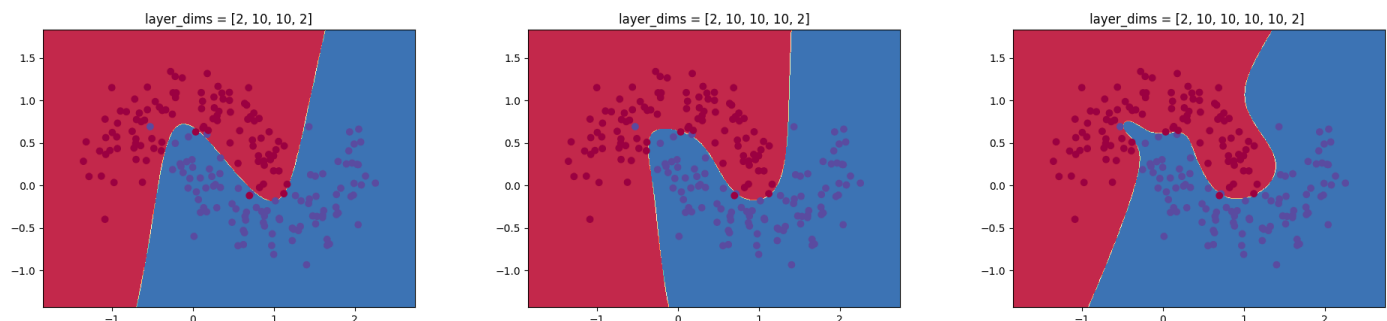
As the number of hidden units increases from 3 to 20, the complexity of the decision boundary also increases. When the hidden unit count is relatively small (e.g., 5), the model struggles to capture the more intricate patterns in the data, leading to underfitting, as shown by the rough and jagged decision boundary. As the hidden unit count grows (e.g., 10 to 20), the decision boundary becomes more flexible and is better able to capture non-linearities in the data. This results in a smoother boundary that better separates the data points.

However, with a very high number of hidden units (e.g., 20), the model may start to overfit, as it becomes sensitive to minor variations in the data. This can be seen in the more complex decision boundary that begins to wrap tightly around some data points, potentially fitting the noise in the dataset.

1 f) Even More Fun - Training a Deeper Network!!!

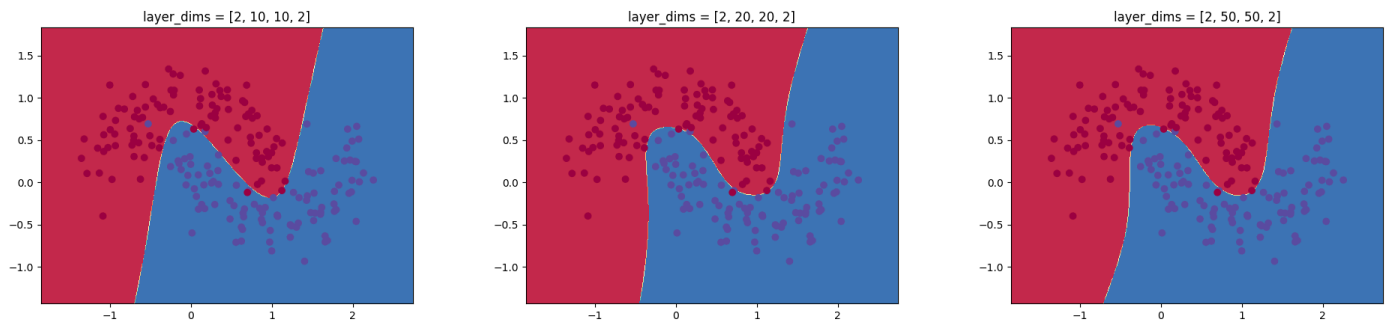
Training network on the `Make_Moons`

Different number of layers



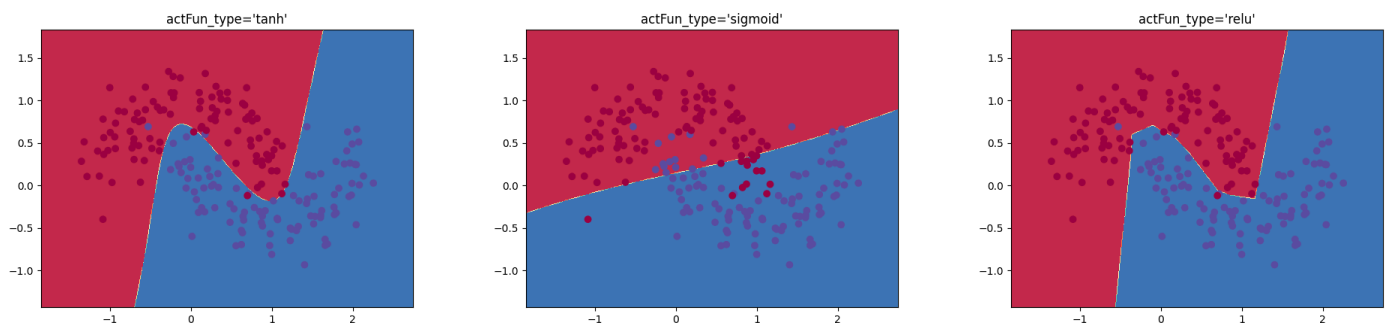
As the number of layers increases, the decision boundary becomes more flexible and complex. The shallow network provides a smoother decision boundary, while deeper networks can fit more detailed patterns but at the cost of potential overfitting. This is seen in the third figure (layer_dims = [2, 10, 10, 10, 10, 2]), where the decision boundary has sharp, intricate changes that likely reflect noise rather than a meaningful pattern.

Different layer sizes



Smaller networks tend to underfit the data, with smoother but less accurate decision boundaries. Moderate layer sizes (e.g., 20 hidden units) provide a good balance, forming boundaries that better match the data without overfitting. Large layer sizes (e.g., 50 hidden units) risk overfitting, with decision boundaries that may become overly complex and sensitive to noise.

Different activation functions



Tanh provides a smooth and effective decision boundary that captures the moon-shaped structure of the data well.

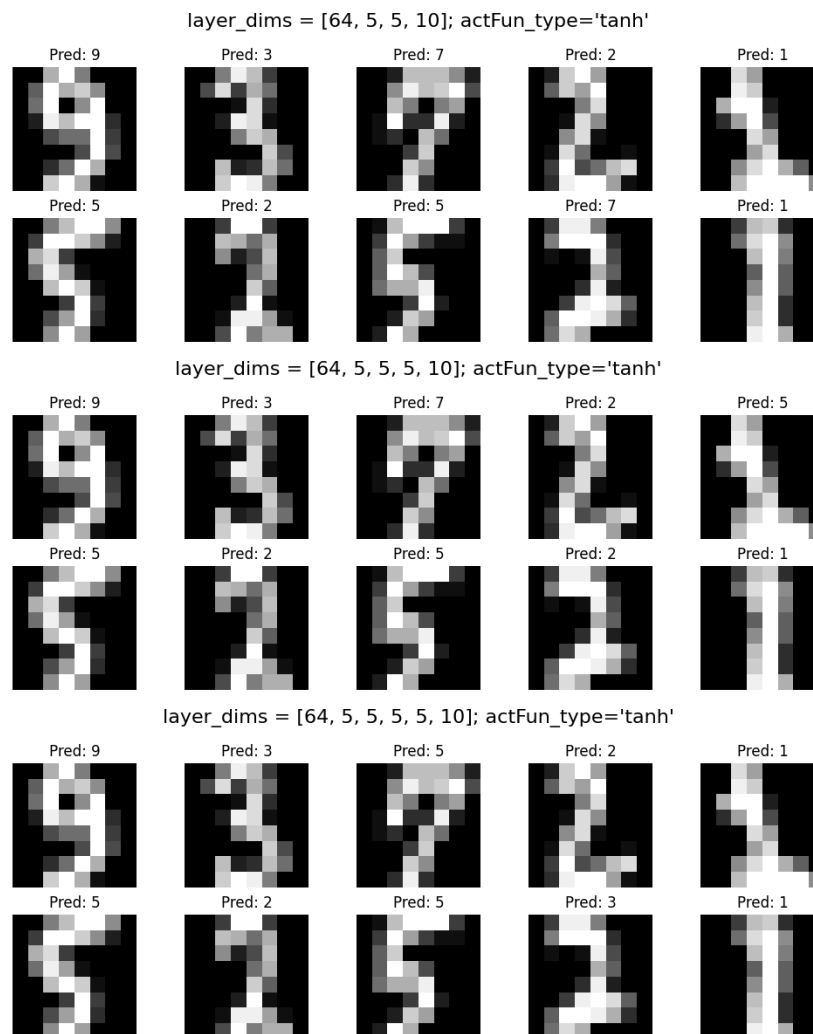
Sigmoid underperforms, offering a more linear boundary that does not fit the data well.

ReLU is quite responsive and introduces sharp edges in the boundary, which may be beneficial for some data but can also lead to overfitting on complex datasets.

Training network on the Optical recognition of handwritten digits dataset

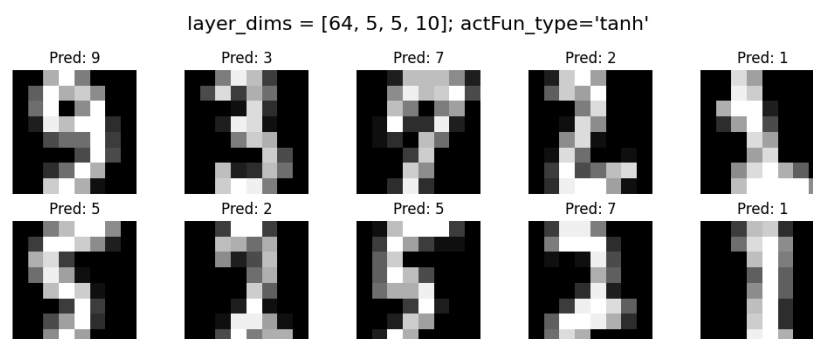
I choose 'Optical recognition of handwritten digits dataset' (https://scikit-learn.org/stable/datasets/toy_dataset.html#optical-recognition-of-handwritten-digits-dataset).

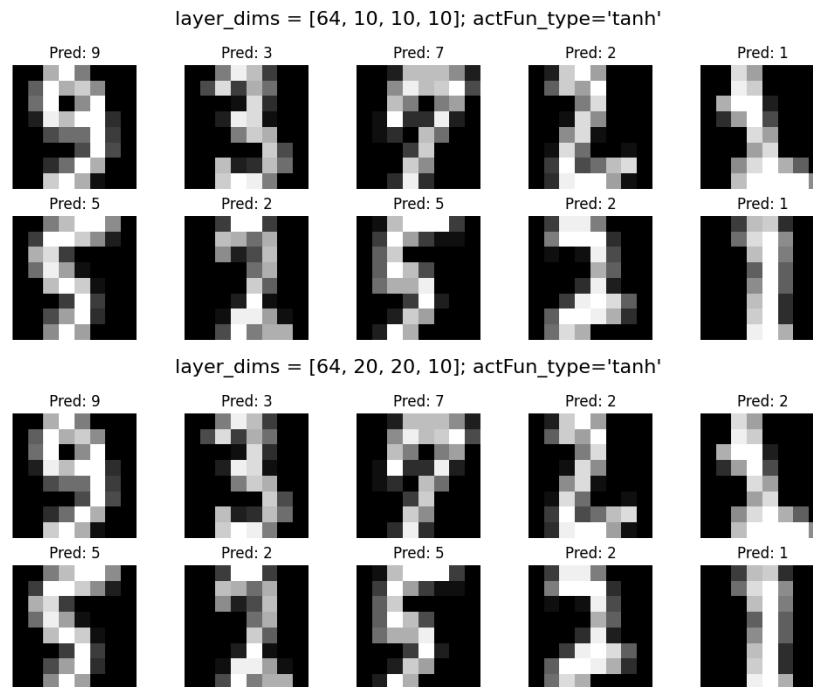
Different number of layers



- Deeper networks tend to capture more intricate patterns, allowing for better classification of more complex digits. This is especially beneficial for datasets like handwritten digits, where subtle differences can distinguish between similar-looking numbers. While deeper networks are often more powerful, increasing layer might cause overfitting (e.g., 2 or 3).

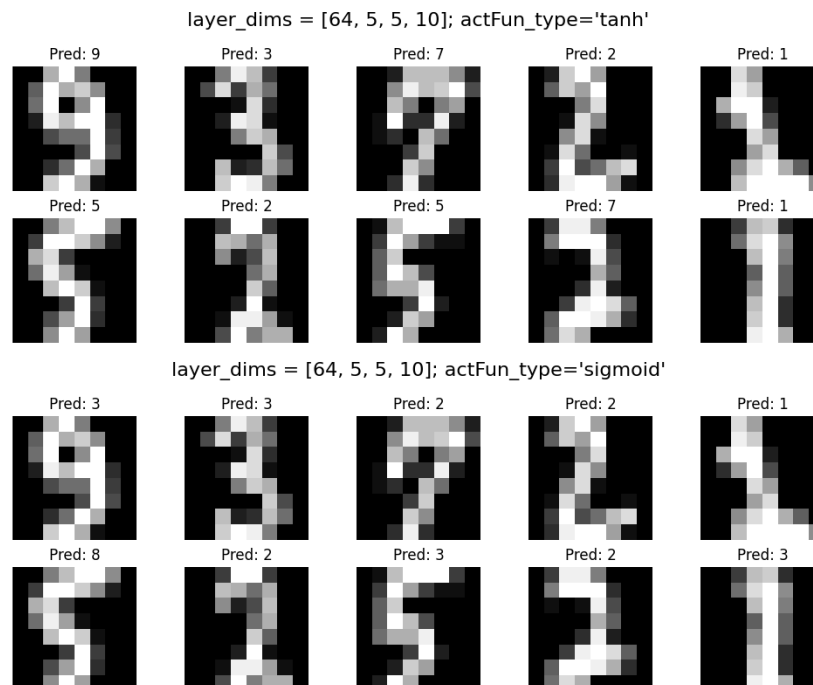
Different layer sizes

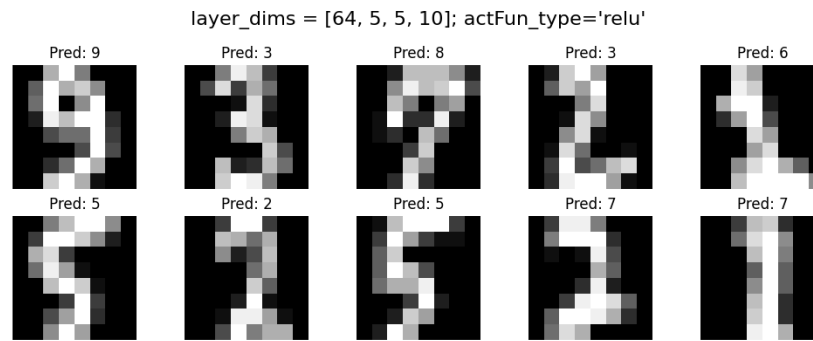




- Larger layer sizes (with more neurons per layer) tend to learn more patterns in the data, especially if the task is complex, like digit recognition. However, if the number of neurons is too small (as in the first example: `layer_dims = [64, 5, 5, 10]`), the network may not have enough capacity to model the data well, leading to underfitting. Overfitting was not evident in this set of comparisons

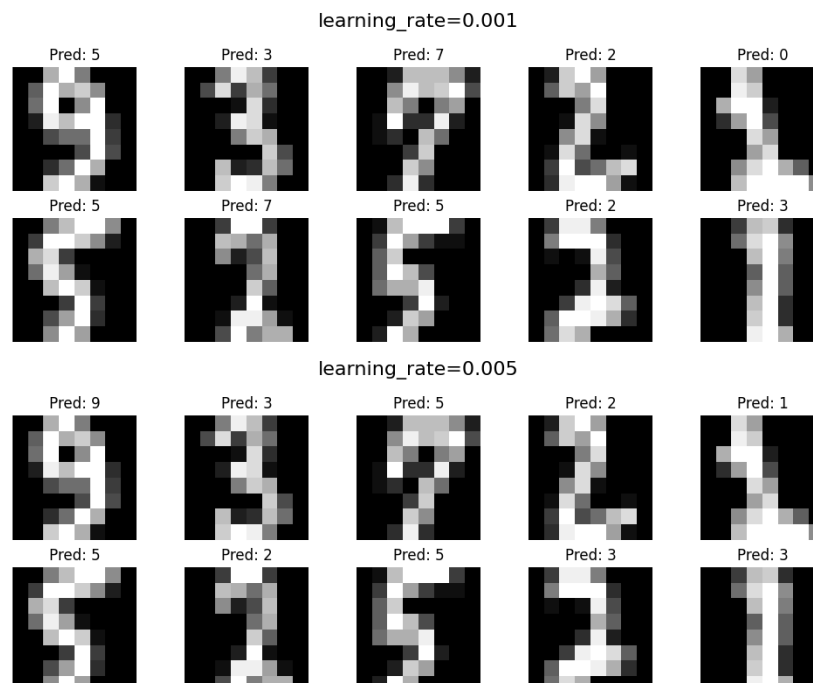
Different activation functions

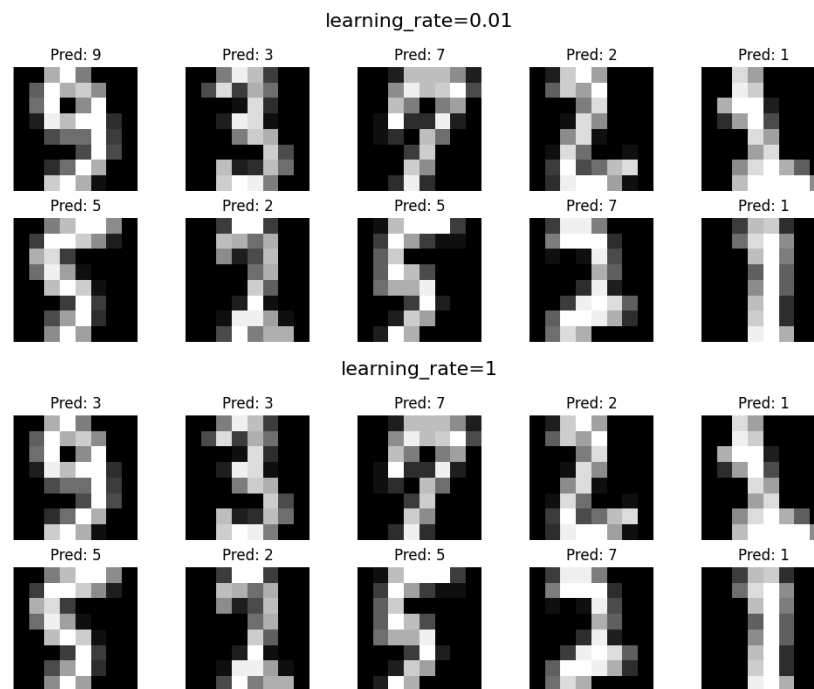




- ReLU: ReLU seems to capture certain patterns well, especially in correctly identifying digits like 9, 3, and 5. However, ReLU can lead to a more inconsistent pattern, as seen with incorrect predictions like labeling a 6 as a 3 and a 2 as a 7. ReLU tends to introduce sharper decision boundaries, which might cause more misclassifications in this context due to its linearity for positive values.
- Sigmoid: Sigmoid activation shows more errors compared to ReLU and Tanh. For example, the digits 9 and 3 are incorrectly classified as 3 and 8, respectively. This is expected as sigmoid functions tend to have a vanishing gradient problem, where the gradients become very small in deep networks, causing slower learning and difficulty in capturing complex patterns.
- Tanh: Tanh shows relatively better performance compared to Sigmoid, with fewer incorrect predictions. The decision boundaries created by Tanh are smoother, making it more suited for this task. In this case, Tanh classified most of the digits correctly, indicating it can handle patterns more effectively in this configuration.

Vary Learning Rate





- As seen in the figures, a low learning rate leads to slower convergence, whereas a very high learning rate causes the model to miss optimal solutions, as demonstrated by the less accurate predictions with learning rate 1. The learning rate of 0.005 or 0.01 shows a more balanced trade-off between speed of convergence and model accuracy.

2. Training a Simple Deep Convolutional Network on MNIST

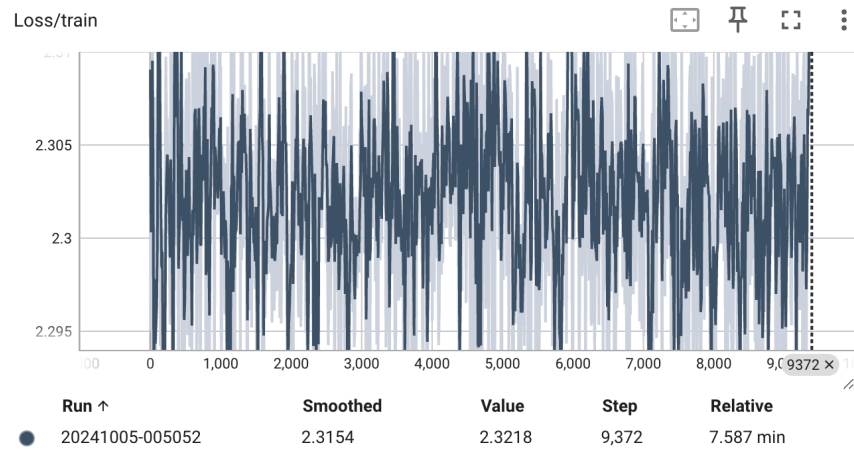
2 a) Build and Train a 4-layer DCN

2.a.5 Run Training

Using the `assignment_1_pytorch_mnist_skeleton.py`, the final test accuracy of my network is 99%.
 Test set: Average loss: 0.0379, Accuracy: 9914/10000 (99%)

2.a.6 Visualize Training

The visualized training loss is:



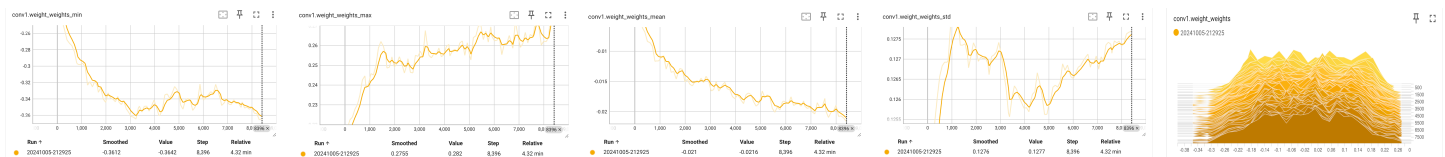
2 b) More on Visualizing Your Training

Here are the statistics (min, max, mean, standard deviation, histogram) of the following terms:
weights, biases, net inputs at each layer, activations after ReLU at each layer, activations after Max-Pooling at each layer.

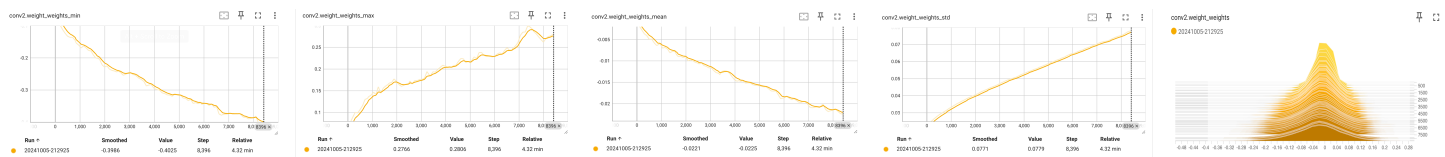
1. Weights
2. Biases
3. Net Inputs at Each Layer
4. Activations after ReLU at Each Layer
5. Activations after Max-Pooling at Each Layer

1. Weights

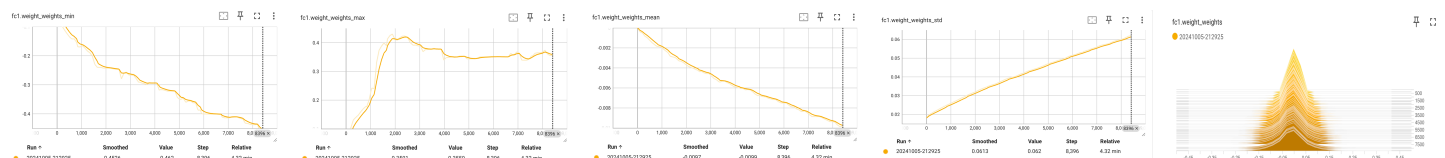
Conv1 Weights



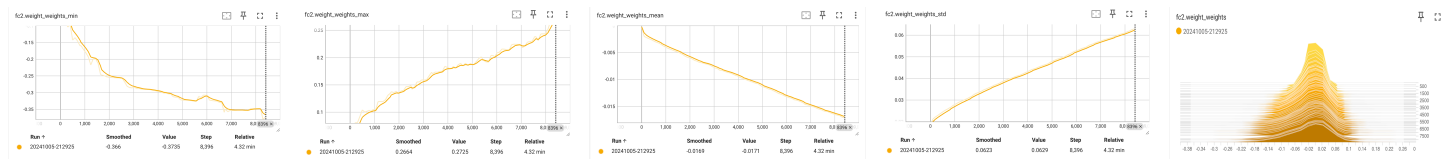
Conv2 Weights



FC1 Weights

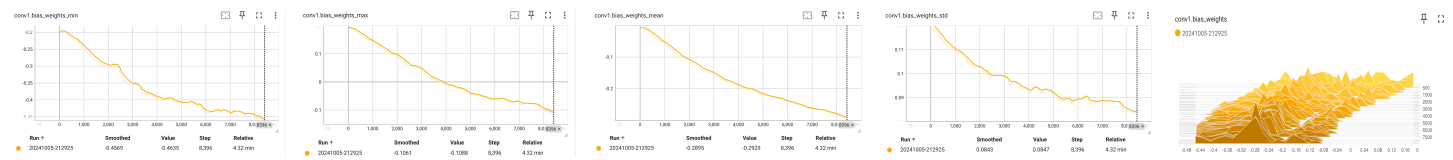


FC2 Weights

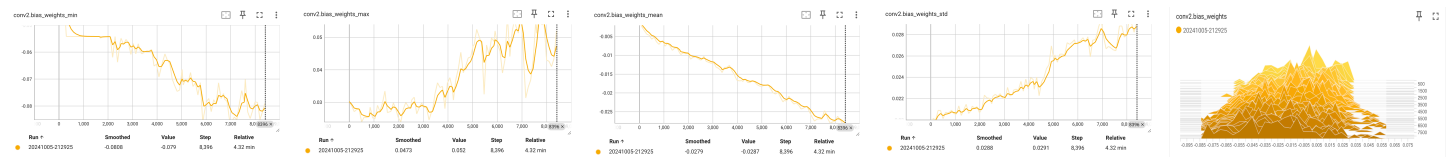


2. Biases

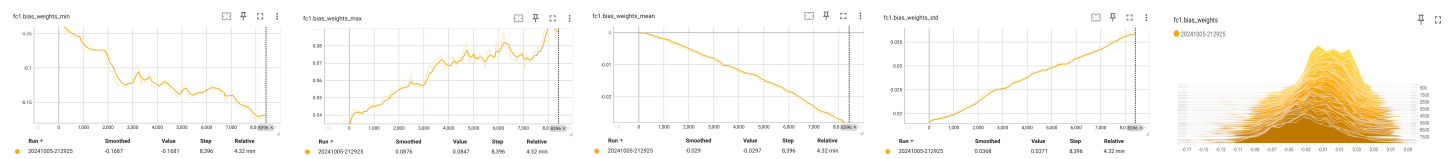
Conv1 Biases



Conv2 Biases



fc1 Biases

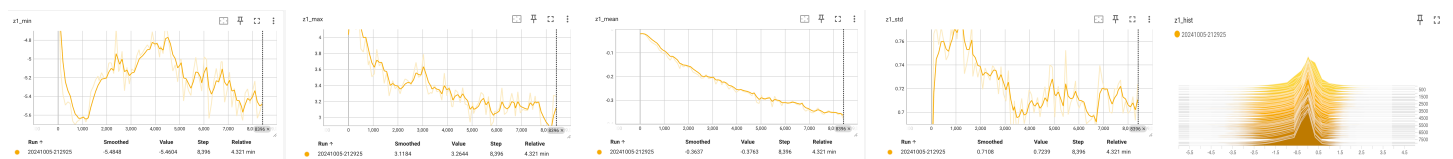


fc2 Biases



3. Net Inputs at Each Layer

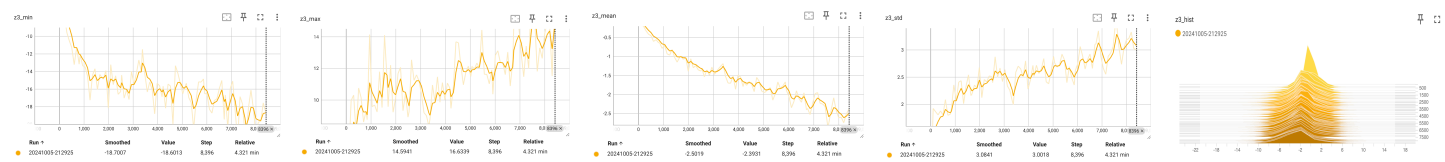
Conv1



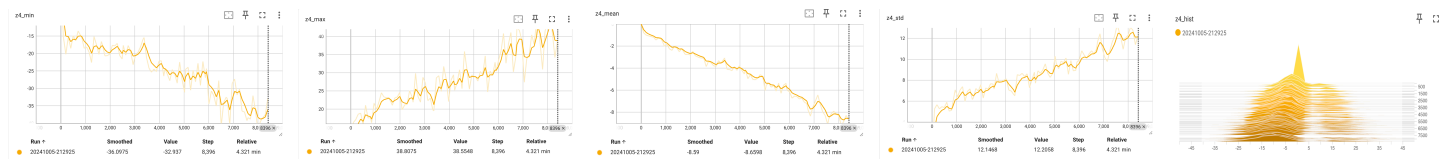
Conv2



FC1

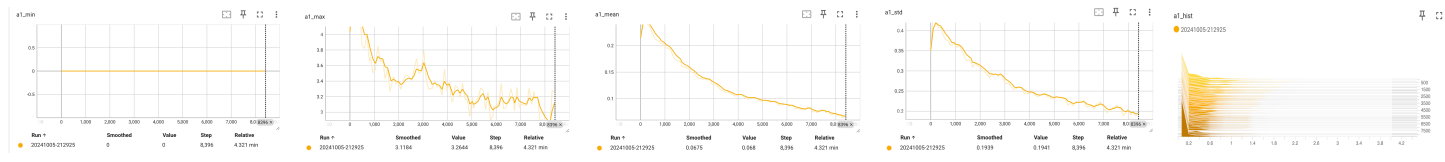


FC2



4. Activations after ReLU at Each Layer

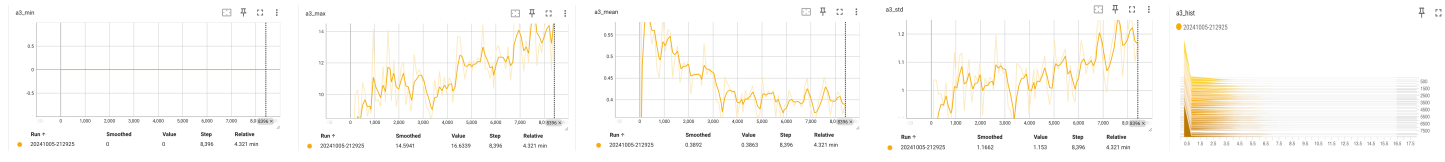
Conv1 (After ReLU)



Conv2 (After ReLU)

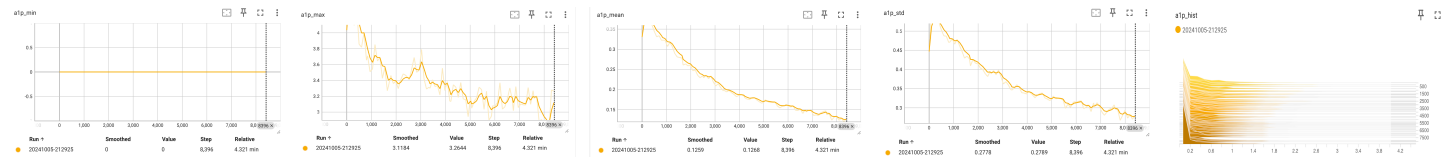


FC1 (After ReLU)

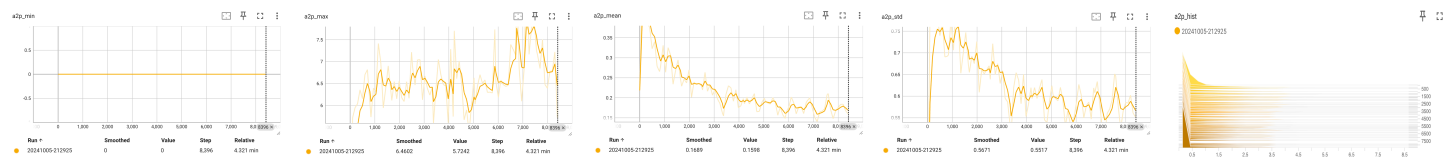


5. Activations after Max-Pooling at Each Layer

Conv1 (After Max-Pooling)



Conv2 (After Max-Pooling)



Test and validation error

Here is the test and validation error after each 1100 iterations and after each epoch.

Train Epoch: 1	[0/54000 (0%)]	Loss: 2.310966	Accuracy: 9.38%
Train Epoch: 1	[6400/54000 (12%)]	Loss: 0.085052	Accuracy: 85.97%
Train Epoch: 1	[12800/54000 (24%)]	Loss: 0.125416	Accuracy: 91.23%
Train Epoch: 1	[19200/54000 (36%)]	Loss: 0.010640	Accuracy: 93.16%
Train Epoch: 1	[25600/54000 (47%)]	Loss: 0.143208	Accuracy: 94.23%
Train Epoch: 1	[32000/54000 (59%)]	Loss: 0.331250	Accuracy: 94.91%
Train Epoch: 1	[38400/54000 (71%)]	Loss: 0.049017	Accuracy: 95.40%
Train Epoch: 1	[44800/54000 (83%)]	Loss: 0.098022	Accuracy: 95.80%
Train Epoch: 1	[51200/54000 (95%)]	Loss: 0.083921	Accuracy: 96.08%

Validation set: Average loss: 0.0648, Accuracy: 5870/6000 (97.83%)

Test set: Average loss: 0.0613, Accuracy: 9798/10000 (97.98%)

Train Epoch: 2	[0/54000 (0%)]	Loss: 0.012937	Accuracy: 100.00%
Train Epoch: 2	[6400/54000 (12%)]	Loss: 0.005945	Accuracy: 98.87%
Train Epoch: 2	[12800/54000 (24%)]	Loss: 0.039993	Accuracy: 98.78%

Validation set: Average loss: 0.0469, Accuracy: 5907/6000 (98.45%)

Test set: Average loss: 0.0404, Accuracy: 9863/10000 (98.63%)

Train Epoch: 2	[19200/54000 (36%)]	Loss: 0.100185	Accuracy: 98.72%
Train Epoch: 2	[25600/54000 (47%)]	Loss: 0.016889	Accuracy: 98.71%
Train Epoch: 2	[32000/54000 (59%)]	Loss: 0.080297	Accuracy: 98.67%
Train Epoch: 2	[38400/54000 (71%)]	Loss: 0.079809	Accuracy: 98.66%
Train Epoch: 2	[44800/54000 (83%)]	Loss: 0.001070	Accuracy: 98.71%
Train Epoch: 2	[51200/54000 (95%)]	Loss: 0.101005	Accuracy: 98.69%

Validation set: Average loss: 0.0351, Accuracy: 5934/6000 (98.90%)

Test set: Average loss: 0.0308, Accuracy: 9907/10000 (99.07%)

Train Epoch: 3	[0/54000 (0%)]	Loss: 0.053644	Accuracy: 98.44%
Train Epoch: 3	[6400/54000 (12%)]	Loss: 0.012697	Accuracy: 99.26%
Train Epoch: 3	[12800/54000 (24%)]	Loss: 0.017919	Accuracy: 99.21%
Train Epoch: 3	[19200/54000 (36%)]	Loss: 0.033121	Accuracy: 99.17%
Train Epoch: 3	[25600/54000 (47%)]	Loss: 0.072393	Accuracy: 99.15%
Train Epoch: 3	[32000/54000 (59%)]	Loss: 0.009029	Accuracy: 99.15%

Validation set: Average loss: 0.0354, Accuracy: 5931/6000 (98.85%)

Test set: Average loss: 0.0305, Accuracy: 9902/10000 (99.02%)

Train Epoch: 3 [38400/54000 (71%)] Loss: 0.000229 Accuracy: 99.12%

Train Epoch: 3 [44800/54000 (83%)] Loss: 0.046839 Accuracy: 99.12%

Train Epoch: 3 [51200/54000 (95%)] Loss: 0.002647 Accuracy: 99.13%

Validation set: Average loss: 0.0320, Accuracy: 5940/6000 (99.00%)

Test set: Average loss: 0.0323, Accuracy: 9907/10000 (99.07%)

Train Epoch: 4 [0/54000 (0%)] Loss: 0.001418 Accuracy: 100.00%

Train Epoch: 4 [6400/54000 (12%)] Loss: 0.000098 Accuracy: 99.50%

Train Epoch: 4 [12800/54000 (24%)] Loss: 0.012301 Accuracy: 99.45%

Train Epoch: 4 [19200/54000 (36%)] Loss: 0.020696 Accuracy: 99.43%

Train Epoch: 4 [25600/54000 (47%)] Loss: 0.010299 Accuracy: 99.42%

Train Epoch: 4 [32000/54000 (59%)] Loss: 0.071617 Accuracy: 99.39%

Train Epoch: 4 [38400/54000 (71%)] Loss: 0.005070 Accuracy: 99.37%

Train Epoch: 4 [44800/54000 (83%)] Loss: 0.021585 Accuracy: 99.34%

Validation set: Average loss: 0.0403, Accuracy: 5930/6000 (98.83%)

Test set: Average loss: 0.0360, Accuracy: 9894/10000 (98.94%)

Train Epoch: 4 [51200/54000 (95%)] Loss: 0.001799 Accuracy: 99.33%

Validation set: Average loss: 0.0436, Accuracy: 5925/6000 (98.75%)

Test set: Average loss: 0.0348, Accuracy: 9894/10000 (98.94%)

Train Epoch: 5 [0/54000 (0%)] Loss: 0.012770 Accuracy: 100.00%

Train Epoch: 5 [6400/54000 (12%)] Loss: 0.000341 Accuracy: 99.61%

Train Epoch: 5 [12800/54000 (24%)] Loss: 0.000604 Accuracy: 99.65%

Train Epoch: 5 [19200/54000 (36%)] Loss: 0.003269 Accuracy: 99.62%

Train Epoch: 5 [25600/54000 (47%)] Loss: 0.005543 Accuracy: 99.58%

Train Epoch: 5 [32000/54000 (59%)] Loss: 0.000382 Accuracy: 99.52%

Train Epoch: 5 [38400/54000 (71%)] Loss: 0.010886 Accuracy: 99.52%

Train Epoch: 5 [44800/54000 (83%)] Loss: 0.003262 Accuracy: 99.50%

Train Epoch: 5 [51200/54000 (95%)] Loss: 0.002068 Accuracy: 99.49%

Validation set: Average loss: 0.0401, Accuracy: 5916/6000 (98.60%)

Test set: Average loss: 0.0382, Accuracy: 9893/10000 (98.93%)

Train Epoch: 6 [0/54000 (0%)] Loss: 0.015430 Accuracy: 100.00%

Train Epoch: 6 [6400/54000 (12%)] Loss: 0.013790 Accuracy: 99.61%

Validation set: Average loss: 0.0473, Accuracy: 5920/6000 (98.67%)

Test set: Average loss: 0.0400, Accuracy: 9891/10000 (98.91%)

Train Epoch: 6 [12800/54000 (24%)] Loss: 0.004235 Accuracy: 99.57%

Train Epoch: 6 [19200/54000 (36%)] Loss: 0.005677 Accuracy: 99.64%

Train Epoch: 6 [25600/54000 (47%)] Loss: 0.007974 Accuracy: 99.63%

Train Epoch: 6 [32000/54000 (59%)] Loss: 0.000180 Accuracy: 99.62%

Train Epoch: 6 [38400/54000 (71%)] Loss: 0.025715 Accuracy: 99.59%

Train Epoch: 6 [44800/54000 (83%)] Loss: 0.026400 Accuracy: 99.57%

Train Epoch: 6 [51200/54000 (95%)] Loss: 0.002099 Accuracy: 99.57%

Validation set: Average loss: 0.0320, Accuracy: 5947/6000 (99.12%)

Test set: Average loss: 0.0352, Accuracy: 9900/10000 (99.00%)

Train Epoch: 7 [0/54000 (0%)] Loss: 0.047212 Accuracy: 98.44%

Train Epoch: 7 [6400/54000 (12%)] Loss: 0.000437 Accuracy: 99.69%

Train Epoch: 7 [12800/54000 (24%)] Loss: 0.000681 Accuracy: 99.64%

Train Epoch: 7 [19200/54000 (36%)] Loss: 0.000547 Accuracy: 99.62%

Train Epoch: 7 [25600/54000 (47%)] Loss: 0.062591 Accuracy: 99.63%

Validation set: Average loss: 0.0305, Accuracy: 5942/6000 (99.03%)

Test set: Average loss: 0.0274, Accuracy: 9918/10000 (99.18%)

Train Epoch: 7 [32000/54000 (59%)] Loss: 0.000124 Accuracy: 99.63%

Train Epoch: 7 [38400/54000 (71%)] Loss: 0.000965 Accuracy: 99.64%

Train Epoch: 7 [44800/54000 (83%)] Loss: 0.001390 Accuracy: 99.62%

Train Epoch: 7 [51200/54000 (95%)] Loss: 0.001378 Accuracy: 99.60%

Validation set: Average loss: 0.0348, Accuracy: 5936/6000 (98.93%)

Test set: Average loss: 0.0326, Accuracy: 9918/10000 (99.18%)

Train Epoch: 8 [0/54000 (0%)]	Loss: 0.002590	Accuracy: 100.00%
Train Epoch: 8 [6400/54000 (12%)]	Loss: 0.022293	Accuracy: 99.81%
Train Epoch: 8 [12800/54000 (24%)]	Loss: 0.026119	Accuracy: 99.81%
Train Epoch: 8 [19200/54000 (36%)]	Loss: 0.003524	Accuracy: 99.72%
Train Epoch: 8 [25600/54000 (47%)]	Loss: 0.004241	Accuracy: 99.69%
Train Epoch: 8 [32000/54000 (59%)]	Loss: 0.015244	Accuracy: 99.72%
Train Epoch: 8 [38400/54000 (71%)]	Loss: 0.001924	Accuracy: 99.70%

Validation set: Average loss: 0.0402, Accuracy: 5940/6000 (99.00%)

Test set: Average loss: 0.0338, Accuracy: 9918/10000 (99.18%)

Train Epoch: 8 [44800/54000 (83%)]	Loss: 0.076407	Accuracy: 99.70%
Train Epoch: 8 [51200/54000 (95%)]	Loss: 0.000034	Accuracy: 99.70%

Validation set: Average loss: 0.0446, Accuracy: 5934/6000 (98.90%)

Test set: Average loss: 0.0493, Accuracy: 9895/10000 (98.95%)

Train Epoch: 9 [0/54000 (0%)]	Loss: 0.000077	Accuracy: 100.00%
Train Epoch: 9 [6400/54000 (12%)]	Loss: 0.000047	Accuracy: 99.80%
Train Epoch: 9 [12800/54000 (24%)]	Loss: 0.003841	Accuracy: 99.73%
Train Epoch: 9 [19200/54000 (36%)]	Loss: 0.001746	Accuracy: 99.69%
Train Epoch: 9 [25600/54000 (47%)]	Loss: 0.003341	Accuracy: 99.68%
Train Epoch: 9 [32000/54000 (59%)]	Loss: 0.001675	Accuracy: 99.68%
Train Epoch: 9 [38400/54000 (71%)]	Loss: 0.003837	Accuracy: 99.71%
Train Epoch: 9 [44800/54000 (83%)]	Loss: 0.054323	Accuracy: 99.72%
Train Epoch: 9 [51200/54000 (95%)]	Loss: 0.000079	Accuracy: 99.72%

Validation set: Average loss: 0.0590, Accuracy: 5926/6000 (98.77%)

Test set: Average loss: 0.0538, Accuracy: 9898/10000 (98.98%)

Train Epoch: 10 [0/54000 (0%)]	Loss: 0.000353	Accuracy: 100.00%
Train Epoch: 10 [6400/54000 (12%)]	Loss: 0.000211	Accuracy: 99.71%

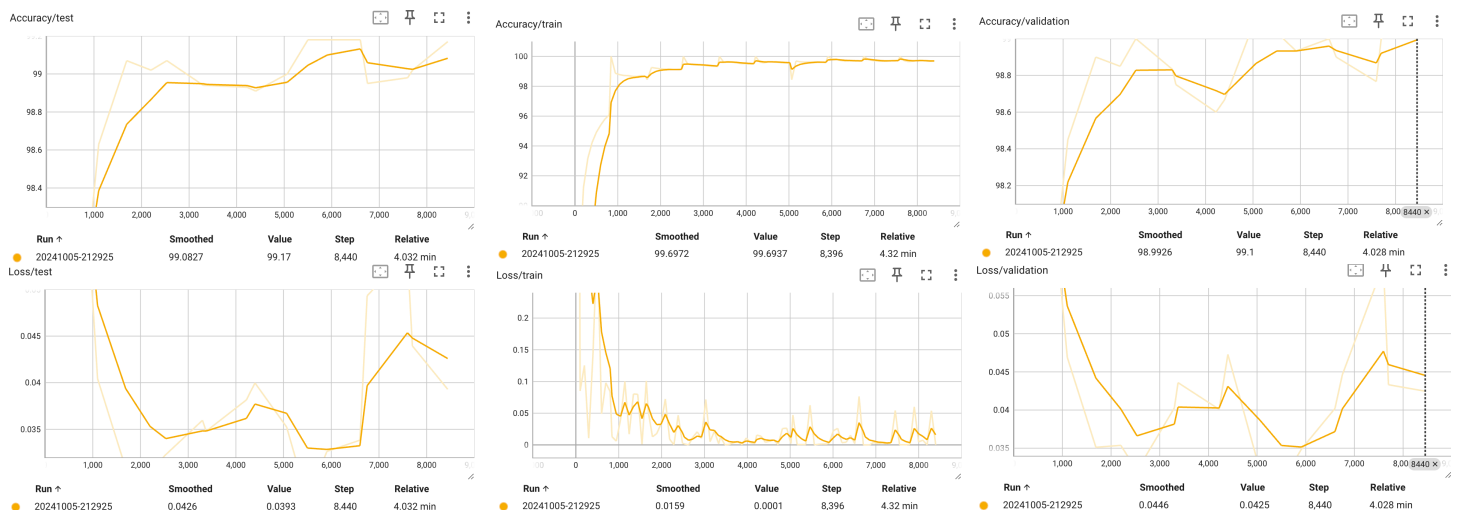
Validation set: Average loss: 0.0433, Accuracy: 5940/6000 (99.00%)

Test set: Average loss: 0.0439, Accuracy: 9902/10000 (99.02%)

Train Epoch: 10 [12800/54000 (24%)]	Loss: 0.000825	Accuracy: 99.65%
Train Epoch: 10 [19200/54000 (36%)]	Loss: 0.060087	Accuracy: 99.68%
Train Epoch: 10 [25600/54000 (47%)]	Loss: 0.004804	Accuracy: 99.73%
Train Epoch: 10 [32000/54000 (59%)]	Loss: 0.007370	Accuracy: 99.73%
Train Epoch: 10 [38400/54000 (71%)]	Loss: 0.000032	Accuracy: 99.70%
Train Epoch: 10 [44800/54000 (83%)]	Loss: 0.053880	Accuracy: 99.67%
Train Epoch: 10 [51200/54000 (95%)]	Loss: 0.000115	Accuracy: 99.69%

Validation set: Average loss: 0.0425, Accuracy: 5946/6000 (99.10%)

Test set: Average loss: 0.0393, Accuracy: 9917/10000 (99.17%)

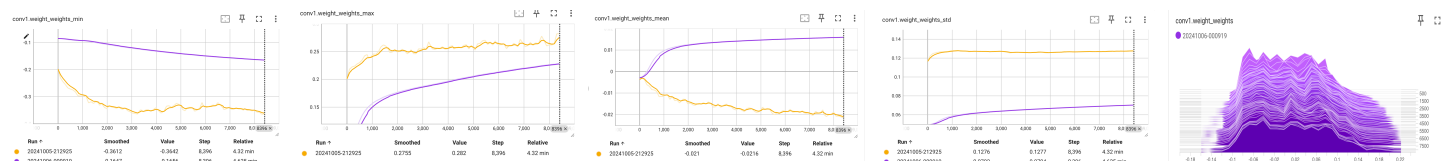


2 c) Time for More Fun!!!

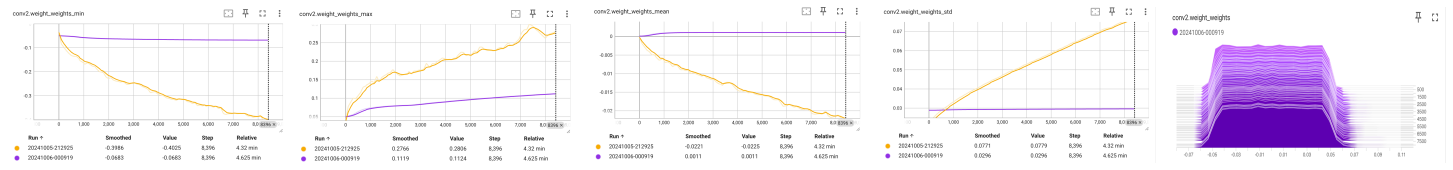
In this part, "**Leaky ReLU + Xavier Initialization + SGD**" is used to replace "ReLU + Random Initialization + Adam Optimizer". In some figures, he monitoring data from the previous section 2 b) is included in this part for comparison purposes.

1. Weights

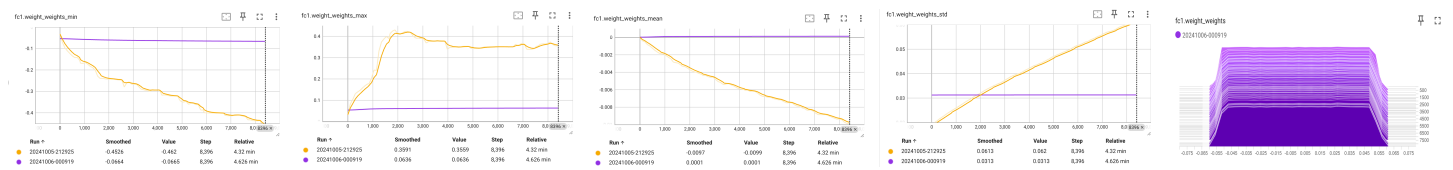
conv1 Weights



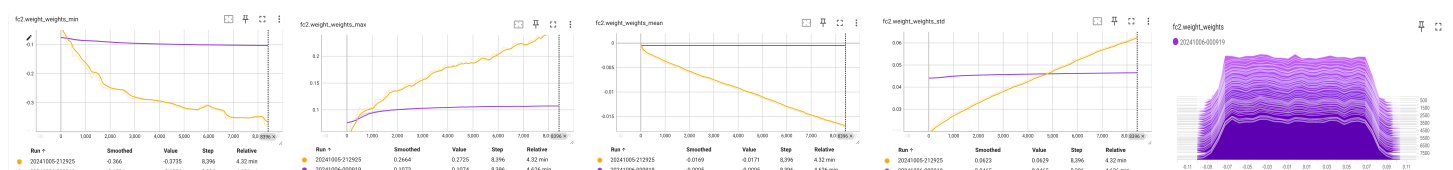
conv2 Weights



fc1 Weights



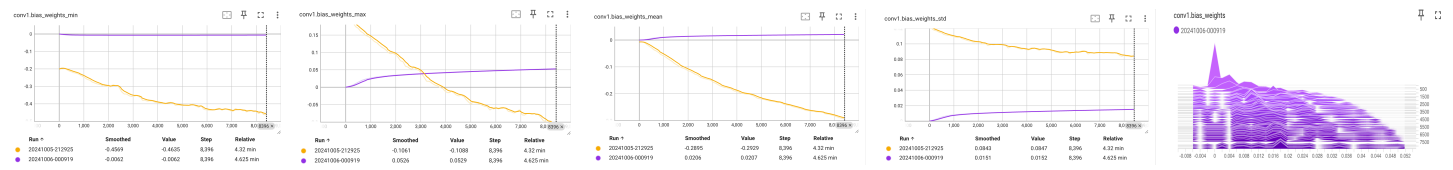
fc2 Weights



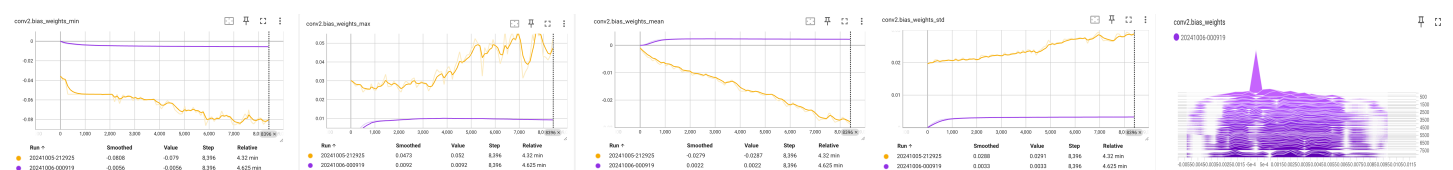
“Leaky ReLU + Xavier Initialization + SGD” offers a more stable and balanced training process. It results in weights that are tightly distributed around zero, with little deviation over time. This can help the network maintain stability, prevent dead neurons, and produce a more robust model that generalizes well to unseen data.

2. Biases

conv1 Biases



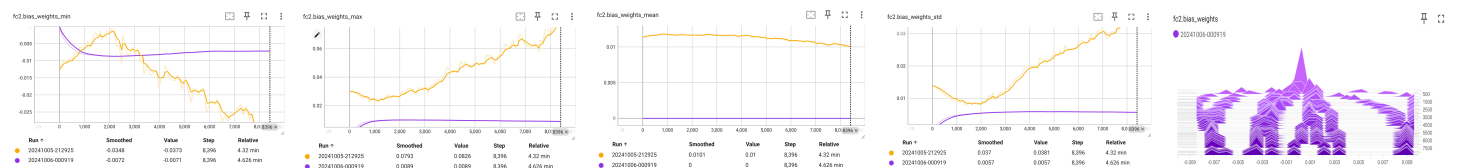
conv2 Biases



fc1 Biases



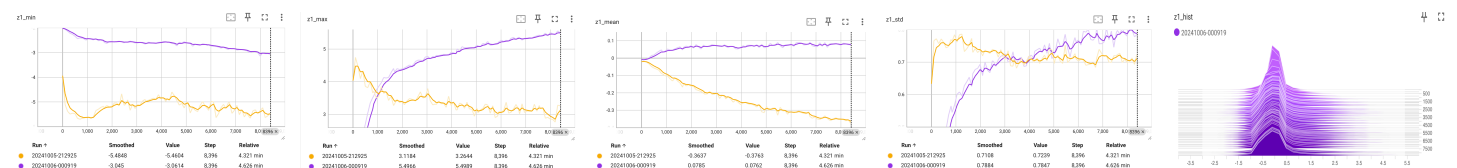
fc2 Biases



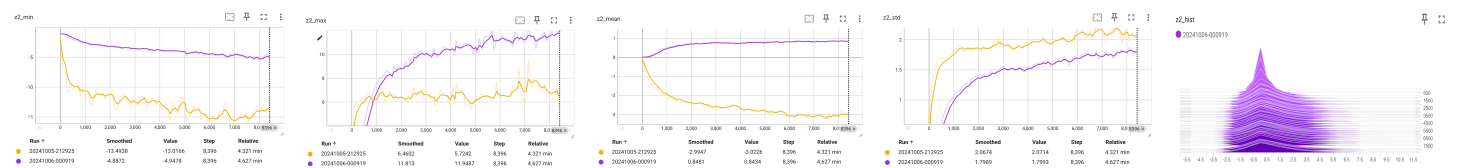
“Leaky ReLU + Xavier Initialization + SGD” leads to slower changes in bias weights, as shown above. Bias values remain clustered around zero without significant spread or change.

3. Net Inputs at Each Layer

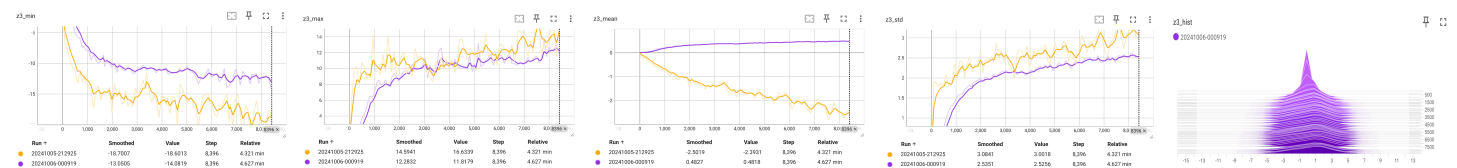
conv1



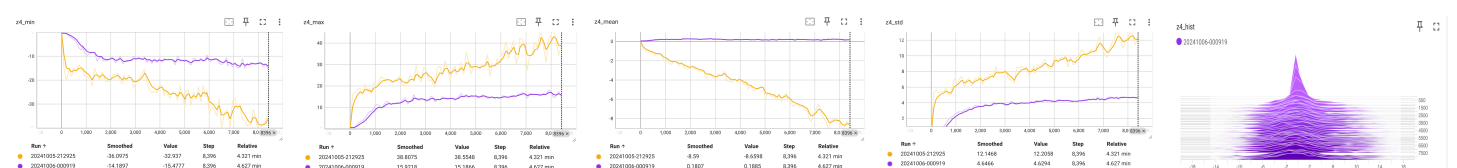
conv2



fc1



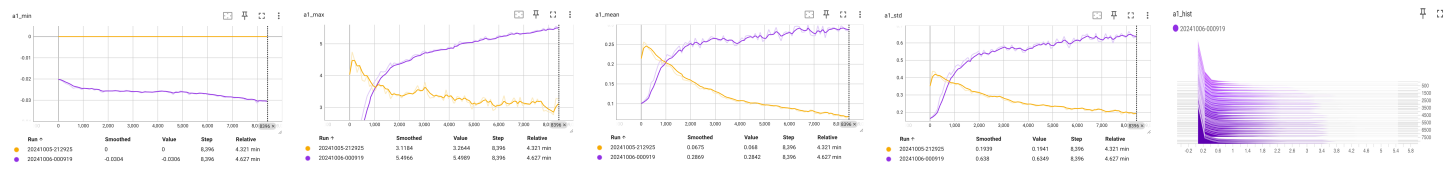
fc2



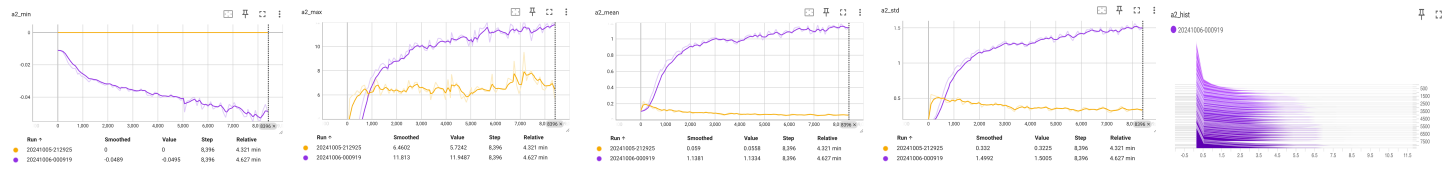
"Leaky ReLU + Xavier Initialization + SGD" demonstrates better stability and more favorable distribution of net inputs throughout the training process. This stability can significantly improve model convergence and overall performance compared to "ReLU + Random Initialization + Adam", which shows potential signs of gradient issues and lack of stable propagation of information through deeper layers.

4. Activations after ReLU at Each Layer

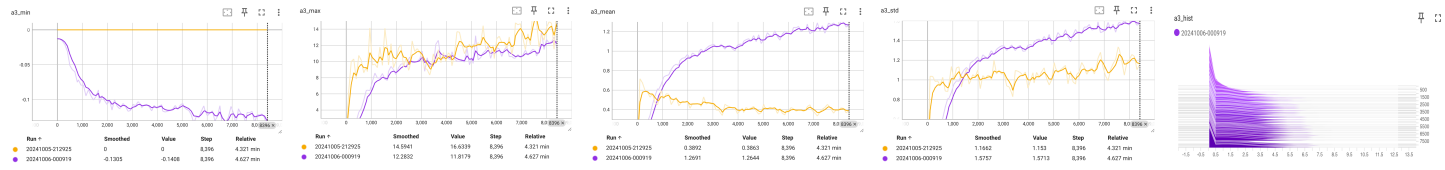
conv1 (After ReLU)



conv2 (After ReLU)



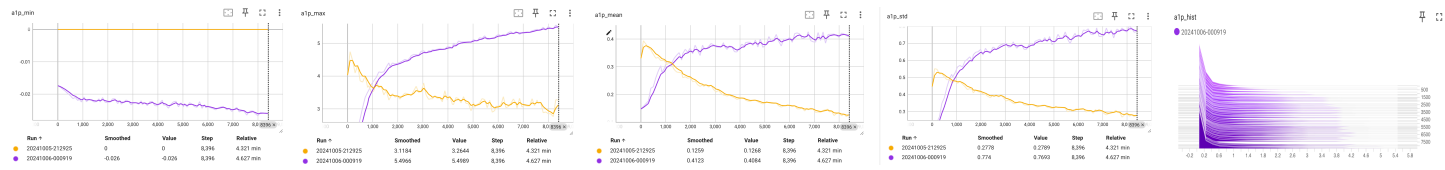
fc1 (After ReLU)



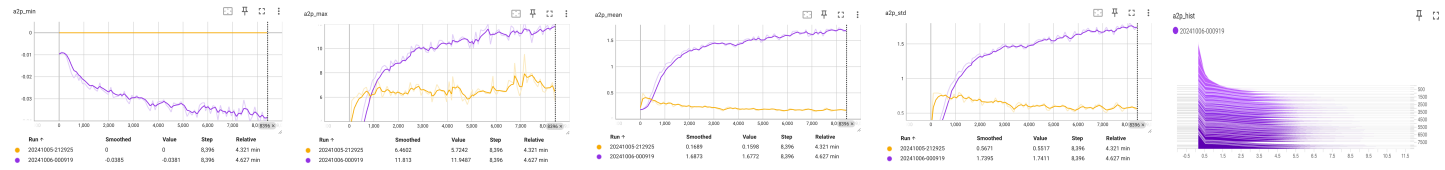
“Leaky ReLU + Xavier Initialization + SGD” appears more balanced, with better-distributed net inputs and activations that suggest a more effective learning process. This could lead to a network that converges faster and generalizes better on unseen data, as it utilizes a larger fraction of its neurons throughout the training process.

5. Activations after Max-Pooling at Each Layer

conv1 (After Max-Pooling)



conv2 (After Max-Pooling)



“Leaky ReLU + Xavier Initialization + SGD” demonstrates a more active and distributed set of neurons during training. Its mean and standard deviation are consistently higher, reflecting better weight initialization and more effective gradient flow during training. It appears advantageous for maintaining a richer and more distributed activation profile, which could lead to better generalization and faster convergence during training. This setup ensures that the gradients are well propagated, avoiding the issue of "dead neurons" that is more prevalent with ReLU and poor initialization strategies.

Test and validation error

Here is the test and validation error after each 1100 iterations and after each epoch.

Train Epoch: 1	[0/54000 (0%)]	Loss: 2.270835	Accuracy: 15.62%
Train Epoch: 1	[6400/54000 (12%)]	Loss: 2.132554	Accuracy: 27.60%
Train Epoch: 1	[12800/54000 (24%)]	Loss: 1.924575	Accuracy: 40.96%
Train Epoch: 1	[19200/54000 (36%)]	Loss: 1.764415	Accuracy: 48.71%
Train Epoch: 1	[25600/54000 (47%)]	Loss: 1.424879	Accuracy: 54.25%
Train Epoch: 1	[32000/54000 (59%)]	Loss: 1.080424	Accuracy: 58.73%
Train Epoch: 1	[38400/54000 (71%)]	Loss: 0.900623	Accuracy: 62.57%
Train Epoch: 1	[44800/54000 (83%)]	Loss: 0.551285	Accuracy: 65.68%
Train Epoch: 1	[51200/54000 (95%)]	Loss: 0.670365	Accuracy: 68.24%

Validation set: Average loss: 0.5334, Accuracy: 5209/6000 (86.82%)

Test set: Average loss: 0.4987, Accuracy: 8832/10000 (88.32%)

Train Epoch: 2	[0/54000 (0%)]	Loss: 0.385433	Accuracy: 95.31%
Train Epoch: 2	[6400/54000 (12%)]	Loss: 0.349048	Accuracy: 88.61%
Train Epoch: 2	[12800/54000 (24%)]	Loss: 0.436198	Accuracy: 88.81%

Validation set: Average loss: 0.4211, Accuracy: 5310/6000 (88.50%)

Test set: Average loss: 0.3877, Accuracy: 8977/10000 (89.77%)

Train Epoch: 2	[19200/54000 (36%)]	Loss: 0.531735	Accuracy: 88.97%
Train Epoch: 2	[25600/54000 (47%)]	Loss: 0.301236	Accuracy: 89.11%
Train Epoch: 2	[32000/54000 (59%)]	Loss: 0.336744	Accuracy: 89.29%
Train Epoch: 2	[38400/54000 (71%)]	Loss: 0.422100	Accuracy: 89.42%
Train Epoch: 2	[44800/54000 (83%)]	Loss: 0.274910	Accuracy: 89.60%
Train Epoch: 2	[51200/54000 (95%)]	Loss: 0.495743	Accuracy: 89.66%

Validation set: Average loss: 0.3215, Accuracy: 5446/6000 (90.77%)

Test set: Average loss: 0.2915, Accuracy: 9176/10000 (91.76%)

Train Epoch: 3	[0/54000 (0%)]	Loss: 0.249658	Accuracy: 92.19%
Train Epoch: 3	[6400/54000 (12%)]	Loss: 0.250801	Accuracy: 91.66%
Train Epoch: 3	[12800/54000 (24%)]	Loss: 0.204923	Accuracy: 91.61%
Train Epoch: 3	[19200/54000 (36%)]	Loss: 0.177207	Accuracy: 91.58%
Train Epoch: 3	[25600/54000 (47%)]	Loss: 0.315173	Accuracy: 91.59%
Train Epoch: 3	[32000/54000 (59%)]	Loss: 0.193325	Accuracy: 91.78%

Validation set: Average loss: 0.2779, Accuracy: 5513/6000 (91.88%)

Test set: Average loss: 0.2493, Accuracy: 9286/10000 (92.86%)

Train Epoch: 3 [38400/54000 (71%)]	Loss: 0.308928	Accuracy: 91.85%
Train Epoch: 3 [44800/54000 (83%)]	Loss: 0.361316	Accuracy: 91.93%
Train Epoch: 3 [51200/54000 (95%)]	Loss: 0.174072	Accuracy: 92.05%

Validation set: Average loss: 0.2577, Accuracy: 5534/6000 (92.23%)

Test set: Average loss: 0.2316, Accuracy: 9343/10000 (93.43%)

Train Epoch: 4 [0/54000 (0%)]	Loss: 0.178001	Accuracy: 95.31%
Train Epoch: 4 [6400/54000 (12%)]	Loss: 0.184532	Accuracy: 93.05%
Train Epoch: 4 [12800/54000 (24%)]	Loss: 0.398939	Accuracy: 92.90%
Train Epoch: 4 [19200/54000 (36%)]	Loss: 0.293146	Accuracy: 93.11%
Train Epoch: 4 [25600/54000 (47%)]	Loss: 0.238584	Accuracy: 93.19%
Train Epoch: 4 [32000/54000 (59%)]	Loss: 0.228268	Accuracy: 93.22%
Train Epoch: 4 [38400/54000 (71%)]	Loss: 0.167429	Accuracy: 93.25%
Train Epoch: 4 [44800/54000 (83%)]	Loss: 0.118116	Accuracy: 93.30%

Validation set: Average loss: 0.2186, Accuracy: 5613/6000 (93.55%)

Test set: Average loss: 0.1983, Accuracy: 9428/10000 (94.28%)

Train Epoch: 4 [51200/54000 (95%)]	Loss: 0.184901	Accuracy: 93.38%
------------------------------------	----------------	------------------

Validation set: Average loss: 0.2215, Accuracy: 5608/6000 (93.47%)

Test set: Average loss: 0.1988, Accuracy: 9416/10000 (94.16%)

Train Epoch: 5 [0/54000 (0%)]	Loss: 0.209816	Accuracy: 95.31%
Train Epoch: 5 [6400/54000 (12%)]	Loss: 0.212075	Accuracy: 94.14%
Train Epoch: 5 [12800/54000 (24%)]	Loss: 0.175676	Accuracy: 94.10%
Train Epoch: 5 [19200/54000 (36%)]	Loss: 0.070757	Accuracy: 94.22%
Train Epoch: 5 [25600/54000 (47%)]	Loss: 0.373567	Accuracy: 94.00%
Train Epoch: 5 [32000/54000 (59%)]	Loss: 0.266510	Accuracy: 94.06%
Train Epoch: 5 [38400/54000 (71%)]	Loss: 0.239855	Accuracy: 94.09%
Train Epoch: 5 [44800/54000 (83%)]	Loss: 0.118112	Accuracy: 94.14%
Train Epoch: 5 [51200/54000 (95%)]	Loss: 0.212446	Accuracy: 94.22%

Validation set: Average loss: 0.1917, Accuracy: 5665/6000 (94.42%)

Test set: Average loss: 0.1721, Accuracy: 9500/10000 (95.00%)

Train Epoch: 6 [0/54000 (0%)] Loss: 0.080255 Accuracy: 100.00%

Train Epoch: 6 [6400/54000 (12%)] Loss: 0.136436 Accuracy: 94.89%

Validation set: Average loss: 0.1840, Accuracy: 5671/6000 (94.52%)

Test set: Average loss: 0.1637, Accuracy: 9519/10000 (95.19%)

Train Epoch: 6 [12800/54000 (24%)] Loss: 0.279735 Accuracy: 95.05%

Train Epoch: 6 [19200/54000 (36%)] Loss: 0.114137 Accuracy: 95.00%

Train Epoch: 6 [25600/54000 (47%)] Loss: 0.253253 Accuracy: 94.87%

Train Epoch: 6 [32000/54000 (59%)] Loss: 0.259401 Accuracy: 94.84%

Train Epoch: 6 [38400/54000 (71%)] Loss: 0.152589 Accuracy: 94.91%

Train Epoch: 6 [44800/54000 (83%)] Loss: 0.189074 Accuracy: 94.95%

Train Epoch: 6 [51200/54000 (95%)] Loss: 0.123383 Accuracy: 95.01%

Validation set: Average loss: 0.1722, Accuracy: 5696/6000 (94.93%)

Test set: Average loss: 0.1560, Accuracy: 9536/10000 (95.36%)

Train Epoch: 7 [0/54000 (0%)] Loss: 0.214651 Accuracy: 93.75%

Train Epoch: 7 [6400/54000 (12%)] Loss: 0.108324 Accuracy: 95.73%

Train Epoch: 7 [12800/54000 (24%)] Loss: 0.093065 Accuracy: 95.62%

Train Epoch: 7 [19200/54000 (36%)] Loss: 0.181933 Accuracy: 95.61%

Train Epoch: 7 [25600/54000 (47%)] Loss: 0.185044 Accuracy: 95.57%

Validation set: Average loss: 0.1583, Accuracy: 5727/6000 (95.45%)

Test set: Average loss: 0.1411, Accuracy: 9585/10000 (95.85%)

Train Epoch: 7 [32000/54000 (59%)] Loss: 0.226988 Accuracy: 95.51%

Train Epoch: 7 [38400/54000 (71%)] Loss: 0.241289 Accuracy: 95.50%

Train Epoch: 7 [44800/54000 (83%)] Loss: 0.075177 Accuracy: 95.52%

Train Epoch: 7 [51200/54000 (95%)] Loss: 0.186845 Accuracy: 95.56%

Validation set: Average loss: 0.1513, Accuracy: 5745/6000 (95.75%)

Test set: Average loss: 0.1369, Accuracy: 9608/10000 (96.08%)

Train Epoch: 8 [0/54000 (0%)]	Loss: 0.254655	Accuracy: 90.62%
Train Epoch: 8 [6400/54000 (12%)]	Loss: 0.187875	Accuracy: 95.59%
Train Epoch: 8 [12800/54000 (24%)]	Loss: 0.108174	Accuracy: 95.86%
Train Epoch: 8 [19200/54000 (36%)]	Loss: 0.274853	Accuracy: 95.78%
Train Epoch: 8 [25600/54000 (47%)]	Loss: 0.078483	Accuracy: 95.82%
Train Epoch: 8 [32000/54000 (59%)]	Loss: 0.167157	Accuracy: 95.86%
Train Epoch: 8 [38400/54000 (71%)]	Loss: 0.057565	Accuracy: 95.92%

Validation set: Average loss: 0.1442, Accuracy: 5749/6000 (95.82%)

Test set: Average loss: 0.1284, Accuracy: 9617/10000 (96.17%)

Train Epoch: 8 [44800/54000 (83%)]	Loss: 0.050463	Accuracy: 95.96%
Train Epoch: 8 [51200/54000 (95%)]	Loss: 0.119095	Accuracy: 95.94%

Validation set: Average loss: 0.1382, Accuracy: 5754/6000 (95.90%)

Test set: Average loss: 0.1250, Accuracy: 9623/10000 (96.23%)

Train Epoch: 9 [0/54000 (0%)]	Loss: 0.131309	Accuracy: 93.75%
Train Epoch: 9 [6400/54000 (12%)]	Loss: 0.220784	Accuracy: 95.87%
Train Epoch: 9 [12800/54000 (24%)]	Loss: 0.101788	Accuracy: 96.03%
Train Epoch: 9 [19200/54000 (36%)]	Loss: 0.179638	Accuracy: 96.18%
Train Epoch: 9 [25600/54000 (47%)]	Loss: 0.086967	Accuracy: 96.40%
Train Epoch: 9 [32000/54000 (59%)]	Loss: 0.080370	Accuracy: 96.31%
Train Epoch: 9 [38400/54000 (71%)]	Loss: 0.151338	Accuracy: 96.32%
Train Epoch: 9 [44800/54000 (83%)]	Loss: 0.114919	Accuracy: 96.34%
Train Epoch: 9 [51200/54000 (95%)]	Loss: 0.107622	Accuracy: 96.36%

Validation set: Average loss: 0.1296, Accuracy: 5775/6000 (96.25%)

Test set: Average loss: 0.1180, Accuracy: 9652/10000 (96.52%)

Train Epoch: 10 [0/54000 (0%)]	Loss: 0.055151	Accuracy: 98.44%
Train Epoch: 10 [6400/54000 (12%)]	Loss: 0.130433	Accuracy: 96.80%

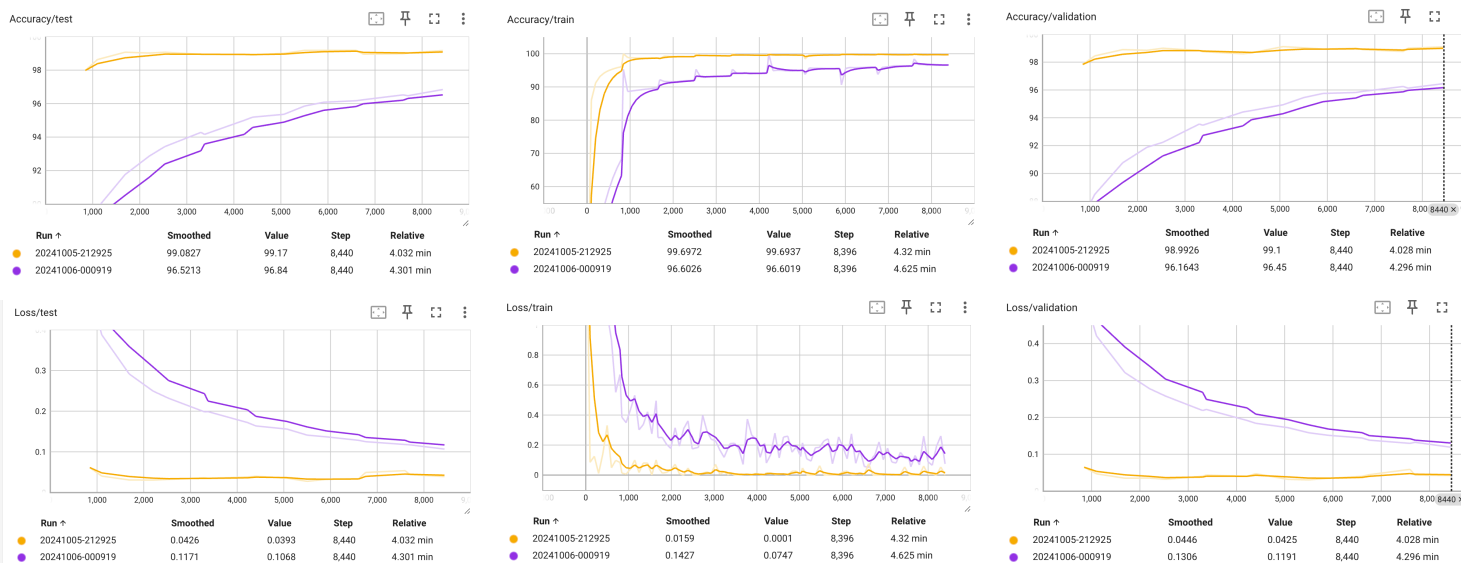
Validation set: Average loss: 0.1324, Accuracy: 5768/6000 (96.13%)

Test set: Average loss: 0.1172, Accuracy: 9647/10000 (96.47%)

Train Epoch: 10 [12800/54000 (24%)]	Loss: 0.067296	Accuracy: 96.75%
Train Epoch: 10 [19200/54000 (36%)]	Loss: 0.259367	Accuracy: 96.61%
Train Epoch: 10 [25600/54000 (47%)]	Loss: 0.088764	Accuracy: 96.64%
Train Epoch: 10 [32000/54000 (59%)]	Loss: 0.064953	Accuracy: 96.58%
Train Epoch: 10 [38400/54000 (71%)]	Loss: 0.192333	Accuracy: 96.56%
Train Epoch: 10 [44800/54000 (83%)]	Loss: 0.260644	Accuracy: 96.57%
Train Epoch: 10 [51200/54000 (95%)]	Loss: 0.074712	Accuracy: 96.60%

Validation set: Average loss: 0.1191, Accuracy: 5787/6000 (96.45%)

Test set: Average loss: 0.1068, Accuracy: 9684/10000 (96.84%)



The "ReLU + Random Initialization + Adam Optimizer" configuration outperformed "Leaky ReLU + Xavier Initialization + SGD" in terms of accuracy, loss convergence, and generalization to validation and test sets. The "ReLU+Adam" model benefited from rapid convergence and higher accuracy due to the adaptive nature of the Adam optimizer, which effectively mitigated issues with random weight initialization and allowed for consistent learning progress. On the other hand, the "Leaky ReLU+SGD" combination struggled with slower convergence and lower accuracy, largely due to the limitations of SGD in finding optimal solutions efficiently, despite using a better weight initialization method (Xavier). Overall, "ReLU + Random Initialization + Adam Optimizer" demonstrated faster training and better overall performance.